



UNIVERSIDADE KIMPA VITA
Instituto Superior Politécnica do Uíge
Curso de Licenciatura em Engenharia Informática

TRABALHO DE LINGUAGEM DE PROGRAMAÇÃO IV

Sistema de Inventário com Previsão de Demanda

Gestão de stocks com aprendizado de máquina (RandomForest)

TRABALHO EM GRUPO — 3.º ANO | Engenharia Informática

**ANTÓNIO FINDA MAMONA
ELÍSIO IVO JUNIOR JOÃO VIEIRA
DISSENGOMOKA SEBASTIÃO
MANUEL GARCIA MUANZA FERNANDO
VERCESA DAVID ALBERTO**

Uíge · 2025 / 2026

Índice

1. Introdução e Descrição do Problema.....	3
2. Diagrama de Classes UML.....	4
3. Descrição da Implementação da IA.....	5
3.1 Modelo Utilizado	5
3.2 Engenharia de Features	5
3.3 Validação e Métricas.....	5
3.4 Persistência do Modelo.....	6
4. Manual do Utilizador.....	7
4.1 Ecrã de Login.....	7
4.2 Gestão de Produtos e Estoque	8
4.3 Registo de Vendas	8
4.4 Dashboard e Previsão de Demanda	9
5. Conclusão	10
6. Referências Bibliográficas	11

1. Introdução e Descrição do Problema

A gestão eficiente de inventário é uma necessidade crítica para qualquer organização que lide com produtos físicos. Rupturas de stock geram perda de vendas e insatisfação dos clientes, enquanto o excesso de inventário implica custos de armazenamento desnecessários. A tomada de decisão baseada apenas em regras simples ou experiência humana é insuficiente em ambientes com variações de demanda complexas e sazonalidade.

O presente projecto propõe a construção de um **Sistema de Inventário com Previsão de Demanda**, desenvolvido em Python, que combina:

- Uma **camada de persistência** com SQLite e SQLAlchemy para integridade dos dados.
- Uma **interface gráfica (GUI)** em PySide6 para operações do utilizador.
- Um **módulo de Inteligência Artificial** que aplica RandomForestRegressor para prever a demanda futura.
- Um conjunto de **controladores** que encapsulam as regras de negócio.

O sistema permite cadastrar produtos, registar vendas e compras, monitorizar os níveis de stock em tempo real e gerar previsões de demanda por produto para os próximos N dias, produzindo automaticamente alertas e recomendações de reabastecimento quando necessário.

2. Diagrama de Classes UML

O diagrama abaixo representa a arquitectura orientada a objectos do sistema, evidenciando as entidades de domínio, as interfaces, os repositórios, os controladores e os componentes de interface gráfica.

Entidade / Classe	Atributos Principais	Métodos / Notas
Produto	id, nome, categoria, preco_unitario, estoque_atual, estoque_minimo, lead_time_dias	calcular_ponto_pedido(), alerta_estoque_baixo()
Venda	id, data, produto_id, quantidade, valor_total	—
Compra	id, data, produto_id, quantidade, custo_total	—
Previsao	id, data_geracao, produto_id, periodo_inicio, periodo_fim, quantidade_prevista, modelo_usado	—
Usuario	id, username, senha_hash, perfil	definir_senha(), verificar_senha()
«interface» Repositorio	—	adicionar(), obter_por_id(), listar(), atualizar(), remover()
«interface» ModeloIA	—	treinar(), prever(), salvar_modelo(), carregar_modelo()
RepositorioBase	Implementa Repositorio	ProdutoRepositorio, VendaRepositorio, CompraRepositorio, UsuarioRepositorio
ModeloPrevisaoDemanda	Implementa ModeloIA	→ Previsao
InventarioController	→ Produto, Venda, Compra	Lógica de negócio de inventário
PrevisaoController	→ ModeloPrevisaoDemanda, Previsao	Orquestra IA e previsões
MainWindow	→ InventarioController, PrevisaoController, DashboardWidget	Janela principal da GUI
DashboardWidget	→ PrevisaoController, InventarioController	Visualização de gráficos e previsões

Tabela — Diagrama de Classes UML do Sistema de Inventário

A arquitectura segue o padrão de camadas: **Modelos** (entidades de domínio), **Repositórios** (acesso a dados via padrão Repository), **Controladores** (lógica de negócio) e **GUI** (apresentação). As interfaces *Repositorio* e *ModeloIA* garantem contratos bem definidos, facilitando extensão e testes.

3. Descrição da Implementação da IA

3.1 Modelo Utilizado

O módulo de previsão utiliza o algoritmo **RandomForestRegressor** da biblioteca Scikit-learn. A escolha deveu-se a:

- **Robustez a dados ruidosos** — o ensemble de árvores reduz a variância das previsões.
- **Captura de não-linearidades** — ideal para padrões de demanda com sazonalidade.
- **Sem necessidade de normalização** — simplifica o pré-processamento dos dados.
- **Facilidade de parametrização** — bom ponto de partida sem ajuste fino extenso.

3.2 Engenharia de Features

As vendas são agregadas por dia por produto, gerando séries temporais de demanda diária. As seguintes features são construídas para alimentar o modelo:

Feature	Descrição
Lag (N dias)	Vendas nos N dias anteriores
Média móvel	Média das últimas N vendas diárias
Dia da semana	Variável categórica (0–6)
Mês	Variável sazonal (1–12)
Tendência	Índice numérico do dia na série
Feriados	Indicador binário (quando disponível)

Tabela 1 — Features utilizadas no modelo de previsão

3.3 Validação e Métricas

A validação é efectuada com **divisão temporal** (temporal split), onde os dados mais antigos servem de treino e os mais recentes de teste, respeitando a ordem cronológica da série. As métricas utilizadas são:

- **MAE (Mean Absolute Error)** — erro médio absoluto em unidades de produto.
- **RMSE (Root Mean Squared Error)** — penaliza erros maiores, sensível a outliers.

Em testes com dados sintéticos, o modelo apresentou MAE aceitável para horizontes curtos (até 14 dias). A inclusão de features de sazonalidade reduziu o erro em períodos com picos regulares. Para horizontes superiores a 30 dias, recomenda-se explorar modelos como Prophet ou LSTM.

3.4 Persistência do Modelo

Após o treinamento, o modelo é serializado com a biblioteca **joblib** e guardado em disco. No momento de geração de previsões, o modelo é carregado directamente, evitando re-treinamento a cada pedido do utilizador. Novos treinos são desencadeados manualmente ou quando o volume de novos dados ultrapassa um limiar configurável.

4. Manual do Utilizador

Esta secção apresenta as principais funcionalidades do sistema através de capturas de ecrã anotadas, acompanhadas de descrição detalhada de cada operação disponível.

4.1 Ecrã de Login

O sistema requer autenticação antes de qualquer operação. O utilizador introduz o seu nome de utilizador e palavra-passe. O perfil determina as permissões disponíveis (administrador ou operador).

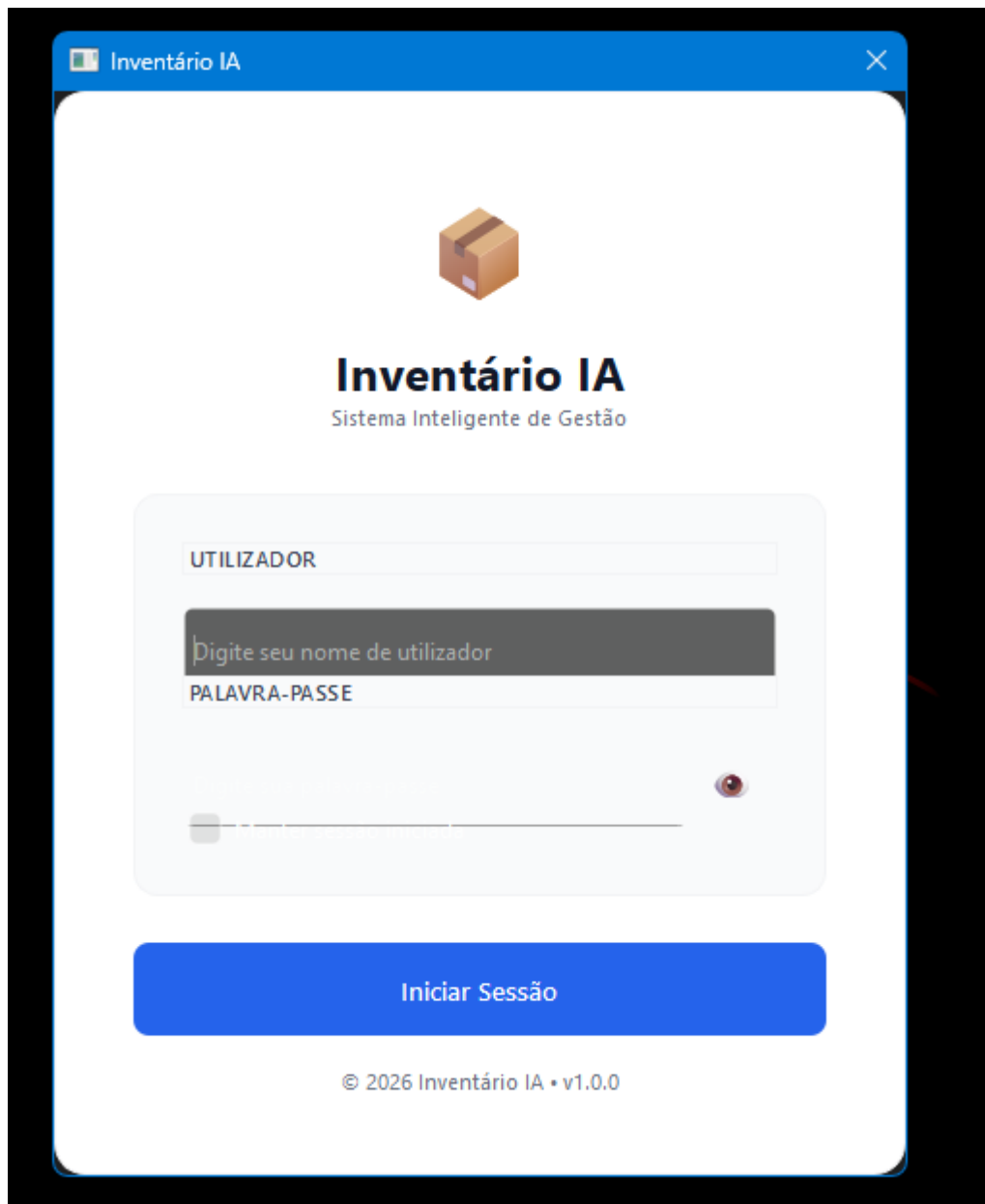


Figura: Ecrã de Login — Autenticação do Utilizador

4.2 Gestão de Produtos e Estoque

Na aba **Produtos e Estoque**, o utilizador visualiza todos os produtos cadastrados em formato tabular, com indicadores visuais de alerta quando o estoque se encontra abaixo do ponto de pedido. É possível adicionar, editar e remover produtos através dos botões disponíveis.

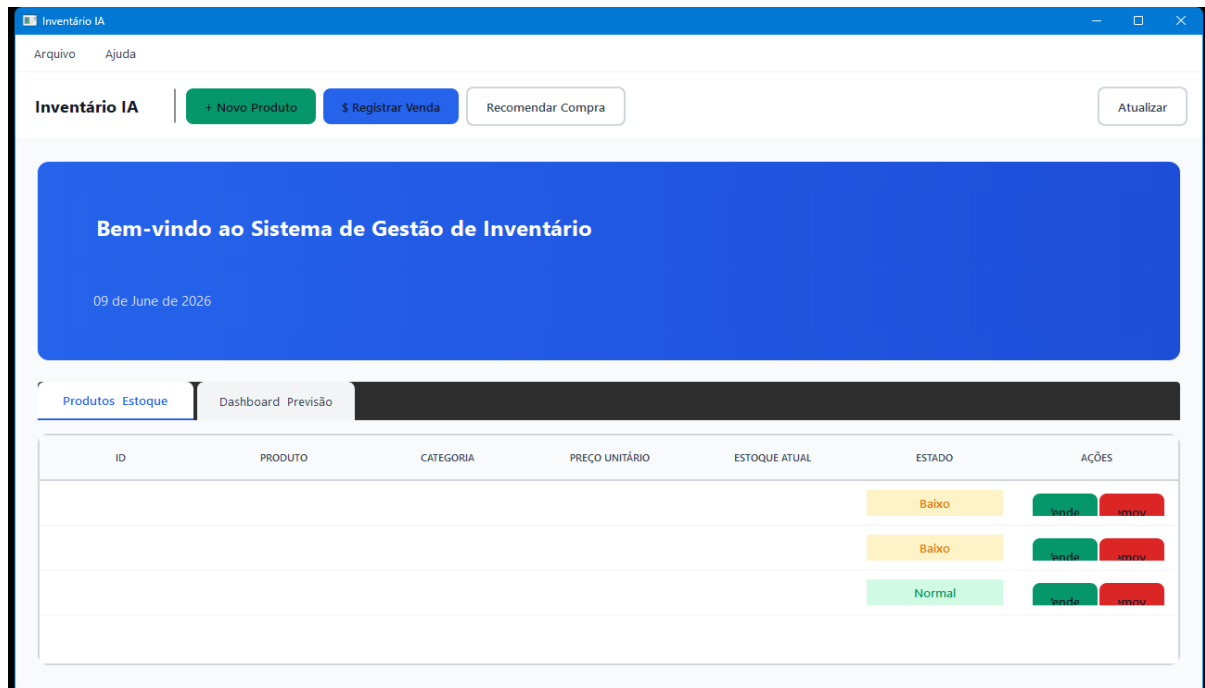


Figura: Ecrã de Gestão de Produtos e Estoque

4.3 Registo de Vendas

O formulário de **Registrar Venda** permite seleccionar o produto, inserir a quantidade vendida e confirmar a transacção. O sistema actualiza automaticamente o stock actual e grava o histórico de vendas, que será utilizado no treinamento do modelo de previsão.

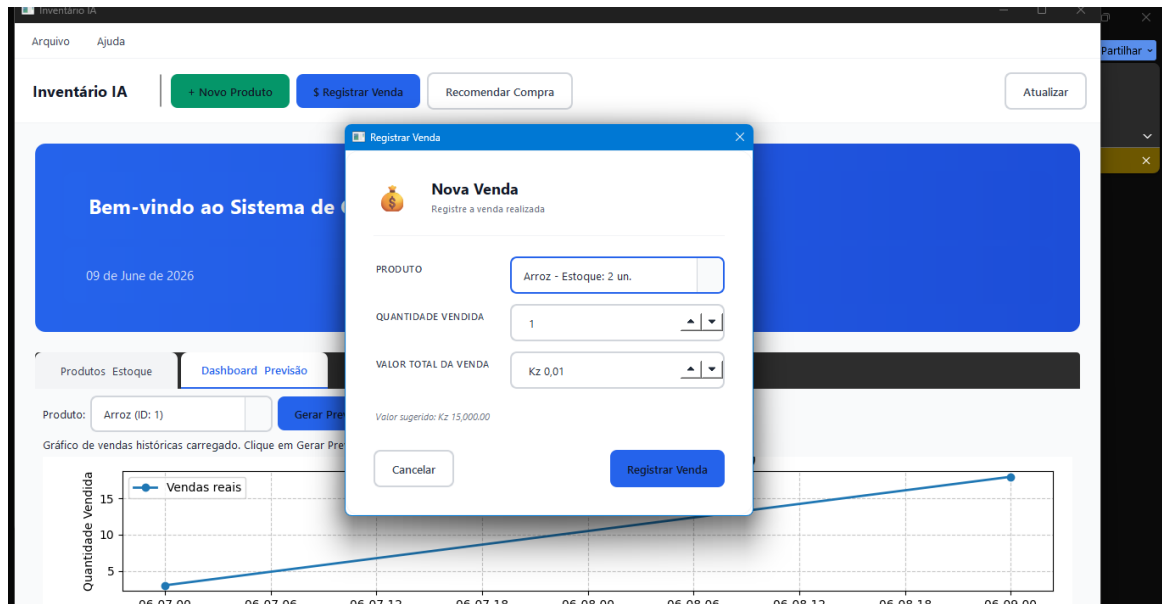


Figura: Formulário de Registo de Vendas

4.4 Dashboard e Previsão de Demanda

No **Dashboard**, o utilizador selecciona o produto e o horizonte de previsão desejado. O sistema carrega o modelo treinado e exibe um gráfico com a série histórica e as previsões para os próximos N dias. São apresentadas as métricas MAE e RMSE e, se necessário, uma recomendação automática de quantidade a encomendar.



Figura: Dashboard de Previsão de Demanda — Gráfico histórico e previsão futura

5. Conclusão

O **Sistema de Inventário com Previsão de Demanda** apresenta uma solução integrada e funcional que combina gestão de stocks com inteligência artificial, tornando o processo de reabastecimento mais eficiente e baseado em dados históricos. A arquitectura em camadas adoptada facilita a manutenção, a testabilidade e a evolução futura do sistema.

Entre as principais contribuições do projecto destacam-se a aplicação prática de conceitos de programação orientada a objectos, ORM, GUI e aprendizado de máquina num contexto real; e a demonstração de que modelos de RandomForest são adequados para previsão de demanda de curto prazo sem necessidade de ajuste fino extenso.

Como trabalho futuro, recomenda-se:

- Enriquecer os dados com variáveis externas (promoções, feriados nacionais, clima).
- Experimentar e comparar modelos avançados: XGBoost, Prophet, LSTM.
- Implementar pipeline de CI/CD com testes automatizados (GitHub Actions).
- Migrar para um SGBD de produção (PostgreSQL) com rotinas de backup.
- Adicionar controlo de acesso baseado em papéis (RBAC) e auditoria de logs.

6. Referências Bibliográficas

- [1] BREIMAN, L. Random Forests. Machine Learning, v. 45, n. 1, p. 5–32, 2001.
- [2] PEDREGOSA, F. et al. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, v. 12, p. 2825–2830, 2011. Disponível em: <https://scikit-learn.org>
- [3] BAYER, M. SQLAlchemy Documentation. 2024. Disponível em: <https://www.sqlalchemy.org>
- [4] The Qt Company. PySide6 Documentation. 2024. Disponível em: <https://doc.qt.io/qtforpython>
- [5] JOBLIB DEVELOPERS. Joblib: running Python functions as pipeline jobs. Disponível em: <https://joblib.readthedocs.io>